

Relazione Tecnica

Distributed Service Registry con Replica Gossip

Corso di Sistemi Distribuiti e Cloud Computing (SDCC)

I. INTRODUZIONE E OBIETTIVI

Nel moderno panorama dello sviluppo software, l'architettura a microservizi è diventata lo standard de facto per la progettazione di sistemi scalabili e resilienti. Tuttavia, la natura dinamica ed effimera delle istanze di servizio introduce una sfida fondamentale: il *service discovery*. I nodi di calcolo possono essere avviati, interrotti o migrati frequentemente, rendendo obsoleti i tradizionali approcci di configurazione statica degli indirizzi di rete.

Il presente progetto si propone di realizzare un **registro di servizi distribuito** e decentralizzato, sviluppato in linguaggio Go. Il sistema è progettato per tracciare in tempo reale la presenza, la localizzazione e lo stato di salute (*health status*) delle istanze di microservizi all'interno di un cluster di nodi registry mutuamente cooperanti.

L'obiettivo principale del lavoro è dimostrare come una soluzione architetturale leggera, ispirata ai principi dei sistemi *peer-to-peer* (P2P) e basata su protocolli di *gossip* combinati con meccanismi di sincronizzazione periodica, possa offrire un'alternativa efficiente e resiliente ai registri centralizzati o fortemente consistenti (come Consul o etcd), privilegiando la disponibilità e la tolleranza al partizionamento (proprietà AP del teorema CAP).

Nello specifico, il sistema è stato progettato per garantire e dimostrare i seguenti requisiti fondamentali:

- **Alta disponibilità (Availability) e Fault Tolerance:** Il servizio di discovery deve rimanere operativo e accessibile dai client anche in presenza di fault parziali del cluster, isolamento di singoli nodi o partizionamenti della rete, evitando la presenza di un *single point of failure*.
- **Consistenza eventuale e Convergenza senza Coordinatore:** Attraverso l'adozione di un protocollo epidemico (*gossip*), lo stato del registro deve convergere autonomamente verso una configurazione coerente su tutti i nodi del cluster, escludendo la necessità di algoritmi di consenso a forte consistenza (es. Paxos o Raft) o di un nodo coordinatore (*leader*), riducendo così l'overhead di comunicazione.
- **Self-Healing e Recovery post-crash:** Capacità dei nodi del cluster di recuperare la piena operatività dopo un crash temporaneo, attivando procedure di riallineamento dello stato (*state transfer*) per colmare il gap informativo accumulato durante il periodo di downtime.

- **Semplicità operativa e di Deployment:** Dimostrare che la complessità intrinseca di un sistema distribuito può essere mitigata sul piano operativo mediante l'uso di strumenti di containerizzazione e automazione, nello specifico Docker Compose per l'orchestrazione locale e un Makefile standardizzato per la gestione del ciclo di vita del software (compilazione, testing, esecuzione).

Per raggiungere tali obiettivi, l'architettura del sistema si scompone in due macro-interfacce di comunicazione distinte, implementate tramite il framework gRPC per massimizzare le performance e garantire la tipizzazione dei contratti di rete:

- 1) **API Client-to-Cluster (Service Operations):** Un set di procedure remote esposte ai microservizi per consentire le operazioni di ciclo di vita standard, quali la registrazione iniziale, l'invio periodico di segnali di vitalità (*heartbeat*), la deregistrazione esplicita e la risoluzione degli indirizzi (*discovery*).
- 2) **API Peer-to-Peer (Cluster Cooperation):** Un canale di comunicazione interno e bidirezionale utilizzato esclusivamente dai nodi del registro per scambiarsi informazioni sullo stato globale, propagare gli aggiornamenti e rilevare l'eventuale fallimento di altri peer.

II. CONTESTO, REQUISITI E SCENARIO DI ESECUZIONE

A. Contesto architetturale e modello di deployment

Il sistema è progettato per operare in un ambiente distribuito e parzialmente isolato, simulato localmente attraverso una strategia di containerizzazione basata su **Docker**. L'architettura minima di riferimento per la validazione del protocollo prevede il dispiegamento di tre istanze indipendenti del nodo registry (*Registry Nodes*) e di un client CLI (*Command Line Interface*) di supporto, adibito alla stimolazione delle API e alla verifica dello stato del cluster.

Ogni nodo registry viene eseguito all'interno di un container dedicato, dotato di un proprio spazio di indirizzamento IP e di risorse computazionali isolate. Dal punto di vista software, ciascun peer mantiene in memoria una **vista locale** (una tabella di routing e un registro di servizi) che rappresenta la propria percezione dello stato globale del sistema. Non essendoci una sorgente di verità centralizzata, la coerenza informativa viene garantita esclusivamente dalle interazioni *peer-to-peer*, attraverso le quali i nodi propagano in modo proattivo le modifiche

(nuove registrazioni, variazioni di stato o rimozioni) al resto del cluster mediante dinamiche epidemiche. La scelta di un cluster a tre nodi rappresenta il punto di partenza ottimale per osservare e validare scenari di tolleranza ai guasti e dinamiche di convergenza multi-hop.

B. Requisiti operativi e prerequisiti di sistema

Per garantire la riproducibilità dell'ambiente di esecuzione e azzerare i conflitti di dipendenze software locali ("*it works on my machine*"), il ciclo di vita del progetto è stato interamente standardizzato. L'ambiente ospite (*host OS*) deve soddisfare esclusivamente due prerequisiti fondamentali:

- **Docker Desktop (o Docker Engine con plugin Compose):** Necessario per l'istanziamento del network virtuale isolato e l'orchestrazione dei container. Docker consente di mappare le porte gRPC dei nodi e di simulare in modo realistico la latenza e il ciclo di vita dei processi distribuiti.
- **GNU Make (utility make):** Utilizzato come motore di automazione. Il file `Makefile` incluso nel progetto astrae la complessità dei comandi Docker e Go, fornendo un'interfaccia a riga di comando unificata e dichiarativa per la compilazione, l'inizializzazione e il testing del sistema.

C. Scenario dimostrativo ed evoluzione temporale

La validazione funzionale e la verifica dei requisiti di resilienza del cluster avvengono attraverso l'esecuzione di uno scenario dimostrativo completo, orchestrato in modo deterministico dal target `make trace-cover`. Questo scenario simula l'intero ciclo di vita di un ecosistema distribuito nell'arco di una sessione di esecuzione cronologica, articolandosi nelle seguenti macro-fasi sequenziali:

- 1) **Bootstrap del Cluster:** Inizializzazione della rete virtuale Docker e avvio simultaneo dei tre nodi registry, i quali avviano la procedura di *peer discovery* per stabilire la topologia iniziale della rete di gossip.
- 2) **Registrazione del Servizio:** Il client CLI effettua una chiamata gRPC verso uno specifico nodo (*Gateway Node*), richiedendo l'inserimento di un nuovo microservizio fittizio all'interno del registro.
- 3) **Propagazione e Convergenza Multi-Nodo:** Osservazione del protocollo di gossip in azione. Si verifica il tempo necessario affinché la notifica della nuova registrazione si propaghi in modo asincrono dal nodo gateway ai restanti peer del cluster, portando il sistema a uno stato di consistenza eventuale.
- 4) **Fase di Heartbeat e Monitoraggio:** Simulazione del mantenimento dello stato di attività del microservizio mediante l'invio continuo di segnali di vitalità (*keep-alive*), monitorati dai nodi del registro.
- 5) **Simulazione di Fault (Crash di un nodo):** Interruzione brutale e non concordata (tramite comando `docker stop` o `kill`) di uno dei nodi del cluster per

simulare un guasto hardware o un crash applicativo di rete.

- 6) **Resilienza e Tolleranza ai Guasti:** Verifica del comportamento dei nodi superstiti. Il cluster ridotto deve continuare a servire le richieste di discovery dei client senza interruzioni, dimostrando l'assenza di *single point of failure* e isolando il nodo fallito.
- 7) **Self-Healing e Recovery (Riallineamento dello stato):** Riavvio del nodo precedentemente interrotto. In questa fase si testa la capacità del nodo reimpresso nella rete di accorgersi del gap informativo accumulato e di attivare una procedura di sincronizzazione attiva con i peer sani per ricostruire la propria vista locale aggiornata.
- 8) **Deregistrazione Finale e Teardown:** Rimozione esplicita del servizio dal registro, verifica della propagazione della cancellazione (*tombstone*) e successivo spegnimento pulito dell'intera infrastruttura di container.

III. ARCHITETTURA DEL SISTEMA

La progettazione del sistema adotta un approccio modulare e disaccoppiato, volto a separare nettamente le interfacce di comunicazione di rete, le strutture dati per la memorizzazione dello stato in memoria e i motori logici che governano i protocolli di sincronizzazione. L'architettura interna di ogni singolo nodo è strutturata in quattro macro-blocchi funzionali indipendenti, i quali cooperano scambiandosi informazioni in modo thread-safe.

A. Componenti principali del nodo

- 1) **Nodo Registry (Core Server & Network Interfaces):** Rappresenta il punto di ingresso del processo Go e agisce da orchestratore locale. Questo componente istanzia e gestisce due server gRPC distinti (o un unico server con servizi multipli registrati):
 - *User API Service:* Dedicato a servire le richieste provenienti dai microservizi esterni (registrazione, rinnovo dell'heartbeat, risoluzione degli indirizzi).
 - *Peer API Service:* Dedicato esclusivamente all'interazione tra i nodi del cluster per lo scambio dei vettori di stato e la ricezione dei messaggi di gossip.

La scelta di gRPC su protocollo HTTP/2 permette di sfruttare connessioni persistenti, multiplexing e una serializzazione efficiente tramite Protocol Buffers.

- 2) **Service Store (Local Service Registry):** È la struttura dati in-memory preposta alla memorizzazione dei record relativi ai microservizi attivi. Per garantire la coerenza eventuale ed evitare conflitti durante la propagazione asincrona, ogni record è associato a una **versione logica** (un timestamp o un contatore monotonicamente crescente,

es. *Lamport Timestamp*). Lo Store implementa meccanismi di mutua esclusione (`sync.RWMutex` in Go) per consentire letture concorrenti ad alta frequenza e scritture protette da *race conditions*.

- 3) **Peer Store (Cluster Membership Table)**: È il modulo deputato al mantenimento della topologia del cluster. Conserva l'elenco dei nodi peer noti, i loro indirizzi di rete e il rispettivo stato di vitalità (*Active*, *Suspect*, *Dead*). Il Peer Store fornisce al motore di gossip gli indirizzi dei nodi bersaglio verso cui instradare i messaggi e aggiorna dinamicamente la mappa del cluster in base alle metriche di raggiungibilità rilevate.
- 4) **Runtime Gossip (Synchronization & Lifecycle Engine)**: Rappresenta il cuore algoritmico del progetto. Si tratta di un motore reattivo e periodico, implementato tramite *goroutines* e cicli di *ticker* asincroni, che governa quattro sotto-funzioni critiche:
 - *Bootstrap*: Procedura iniziale di join in cui il nodo contatta i peer noti (*seed nodes*) per integrarsi nel cluster.
 - *Gossip*: Scelta casuale di un sottoinsieme di peer a cui inviare in modo proattivo gli aggiornamenti locali (*rumor mongering*).
 - *Reconcile (Anti-Entropy)*: Meccanismo di background a intervalli più lunghi per confrontare l'intero stato del Service Store con un peer e sanare eventuali disallineamenti passati inosservati.
 - *Pruning & Eviction*: Monitoraggio attivo del timeout dei peer e rimozione (o marcatura come obsoleti tramite *tombstoning*) delle istanze non più raggiungibili.

B. Topologia di rete e modello di persistenza

In fase di esercizio e validazione, la topologia standard prevede il deployment di tre nodi registry posizionati all'interno di una rete virtuale isolata di tipo **Docker bridge**. Questa configurazione consente la risoluzione dei nomi dei container tramite il DNS interno di Docker (es. `registry-1`, `registry-2`, `registry-3`) e simula un ambiente di rete locale (*LAN*) ad alta affidabilità ma soggetto a possibili latenze o partizionamenti software controllati.

Sebbene il registro operi prevalentemente in memoria per massimizzare il throughput delle risposte di discovery, ogni nodo è configurato con un modello di **persistenza locale su volume Docker dedicato**. All'atto di ogni variazione consolidata dello stato (es. nuova registrazione confermata o rimozione), il Service Store effettua il dump asincrono o la scrittura in *Append-Only File* (AOF) dei dati su un file system mappato sul volume host. Questo previene la perdita totale delle informazioni in caso di crash distruttivo del container, consentendo al runtime, in fase di riavvio, di ricaricare lo stato pre-crash prima di avviare la procedura di riconciliazione con i peer.

C. Interfacce gRPC e modello dati

La definizione dei contratti di rete e la serializzazione dei messaggi sono affidate a **Protocol Buffers (v3)**. Questa scelta garantisce un accoppiamento debole tra i moduli, un'efficace validazione dei tipi a tempo di compilazione e performance elevate in termini di throughput e footprint di rete. Il file di specifica `.proto` definisce due servizi gRPC logicamente distinti, mappati sulle due macro-interfacce di comunicazione dell'architettura.

1) Definizione dei Servizi gRPC:

- **ServiceRegistry (Interfaccia Client-to-Cluster)**: Gestisce il ciclo di vita dei microservizi applicativi che intendono fare uso del sistema di discovery. Espone le seguenti procedure remote:
 - **RegisterService**: Consente a una nuova istanza di microservizio di dichiarare la propria presenza, memorizzando le informazioni di rete e i metadati associati nel Service Store del nodo contattato.
 - **DeregisterService**: Avvia la rimozione esplicita e controllata di un'istanza dal registro (es. durante uno shutdown pulito del microservizio).
 - **Heartbeat**: Chiamata ad alta frequenza e basso payload utilizzata dai microservizi per notificare periodicamente la propria vitalità e resettare i timer di timeout sul registro.
 - **GetService / ListServices**: Metodi di consultazione idempotenti utilizzati dai client per risolvere l'indirizzo di uno specifico servizio o per ottenere l'elenco completo delle istanze disponibili nel cluster.
- **RegistryPeer (Interfaccia Peer-to-Peer)**: Modella il protocollo di cooperazione interna e sincronizzazione dello stato tra i diversi nodi del cluster registry.
 - **JoinCluster**: Eseguita da un nodo in fase di bootstrap per annunciare la propria entrata nella rete a un nodo seed noto, richiedendo l'inclusione nel Peer Store.
 - **GossipSync**: Il canale principale del protocollo epidemico. Viene invocata periodicamente per spingere (*push*) in modo asincrono un vettore di aggiornamenti locali verso un sottoinsieme di peer scelti casualmente.
 - **PullState**: Utilizzata durante le fasi di riconciliazione anti-entropia o di recovery post-crash. Consente a un nodo di richiedere esplicitamente l'intero snapshot dello stato di un altro peer per sanare disallineamenti macroscopici.

2) *Modello Dati e Struttura del ServiceRecord*: L'entità informativa fondamentale scambiata sulla rete e mantenuta in memoria è il messaggio **ServiceRecord**. Per supportare la natura distribuita e decentralizzata del sistema, la struttura dati non si limita a tracciare i dettagli

applicativi, ma integra metadati cruciali per la gestione della consistenza eventuale e della tolleranza ai guasti.

I campi principali che compongono il messaggio sono così strutturati:

- **Identificazione univoca:**

- **service_name** (string): Il nome logico del microservizio (es. "auth-service"), utilizzato dai client in fase di discovery per raggruppare istanze omogenee.
- **service_id** (string): Identificativo univoco globale (es. un UUID o un hash string) della singola istanza fisica del microservizio, necessario per distinguerla dai suoi cloni replicati.

- **Metadati applicativi e di rete:**

- **endpoint** (string): L'indirizzo di rete (IP/Host e Porta, es. "10.0.1.5:8080") a cui inviare le richieste di business.
- **version** (string): La versione applicativa del microservizio (es. "v1.2.0"), utile per supportare strategie di routing avanzate come i *canary deployments* o i rilasci *blue-green*.

- **Stato di salute e vitalità:**

- **health_status** (enum): Stato interno dell'istanza (es. HEALTHY, UNHEALTHY, SUSPECT).
- **last_heartbeat_timestamp** (int64): Marcatura temporale dell'ultimo segnale di keep-alive ricevuto dal client, espresso in formato Unix Epoch. Viene monitorato dai nodi per calcolare la scadenza (*TTL*) dell'istanza.

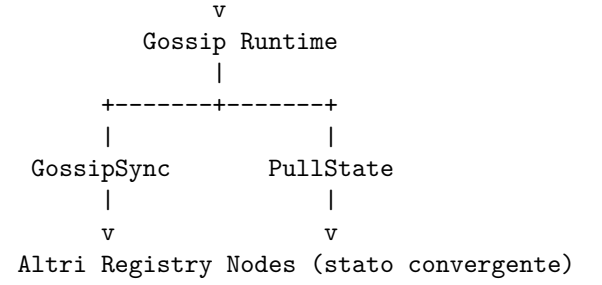
- **Proprietà distribuite e ordinamento:**

- **owner_node_id** (string): L'identificativo del nodo registry che ha preso in carico la registrazione iniziale o che riceve gli heartbeat diretti del servizio. Questo campo permette di stabilire una responsabilità primaria per le operazioni di monitoraggio attivo.
- **logical_version** (uint64): Un contatore monotonicamente crescente associato al record. Questo metadato è fondamentale nelle dinamiche di gossip: agisce come un meccanismo di ordinamento causale semplificato (ispirato ai timestamp di Lamport) per determinare l'anzianità e la validità di un aggiornamento. Quando un nodo riceve un record via gossip, confronta la **logical_version** locale con quella ricevuta: l'aggiornamento viene accettato e sovrascritto solo se la versione ricevuta è strettamente maggiore, risolvendo all'origine i conflitti derivanti da messaggi di rete duplicati o disordinati (*out-of-order delivery*).

D. Flusso logico ad alto livello

Service Instance ---> Registry Node (API ServiceRegistry)

|
| scrive su ServiceStore



IV. ARCHITETTURA, ORCHESTRAZIONE E SINCRONIZZAZIONE DEL CLUSTER

Il ciclo di vita del sistema si basa su un paradigma *Docker-first*, orchestrato tramite automazione nativa GNU Make. Il flusso operativo si divide in due macro-fasi: l'inizializzazione distruttiva dello stato precedente e la convergenza del cluster tracciato.

A. Avvio ed Orchestrazione del Cluster

L'esecuzione del target principale **make trace-up** implementa una sequenza deterministica definita nel Makefile:

$$\text{trace-up} \implies \mathcal{K} \cup \mathcal{B} \cup \mathcal{S} \cup \mathcal{H} \quad (1)$$

Dove ogni operazione atomica è mappata come segue:

- **$\mathcal{K}$ (Reset dello stato):** Esecuzione di **docker compose down -v** (Makefile:104). Rimuove i container attivi, le reti associate e forza l'epurazione dei volumi persistenti anonimi e nominativi, garantendo l'idempotenza dell'avvio.
- **$\mathcal{B}$ (Compilazione e Build):** Generazione parallela delle immagini Docker per i tre nodi di *Registry* e per lo strumento di *CLI* (Makefile:105-106).
- **$\mathcal{S}$ (Inizializzazione dello stato):** Creazione dei file di tracciamento locali sul sistema host per la sincronizzazione dei log (Makefile:107).
- **$\mathcal{H}$ (Attivazione del Demone):** Lancio in background del processo di monitoraggio continuo tramite il target **trace-heartbeat** (Makefile:111), garantendo la vitalità del cluster in regime operativo.

La pulizia ordinaria (senza wipe del volume logico) è delegata a **make compose-down** (Makefile:96), che preserva i dati su disco.

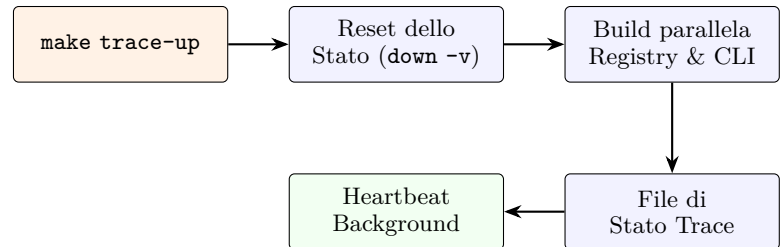


Figura 1. Flusso deterministico di inizializzazione del cluster.

B. Topologia di Docker Compose e Modello di Rete

Il file `docker-compose.yml` definisce un'infrastruttura a quattro servizi logici interconnessi tramite una rete virtuale isolata di tipo *bridge* ($\mathcal{N}_{\text{bridge}}$). Siano $N = \{n_1, n_2, n_3\}$ i tre nodi di Registry e C_{cli} il client effimero. Ciascun nodo esegue lo stesso costruito d'immagine astratta, differenziandosi esclusivamente per il file di configurazione iniettato all'avvio.

La mappatura delle porte tra l'interfaccia di loopback dell'host (P_{host}) e le porte interne ai container ($P_{\text{container}}$) risponde alla seguente funzione di partizionamento:

$$f(n_i) : P_{\text{container}} \rightarrow P_{\text{host}}(n_i) \quad \text{dove} \quad P_{\text{container}} = 50051 \quad (2)$$

$$\begin{cases} f(n_1) \Rightarrow 50051 \rightarrow 50051 \\ f(n_2) \Rightarrow 50051 \rightarrow 50052 \\ f(n_3) \Rightarrow 50051 \rightarrow 50053 \end{cases} \quad (3)$$

L'isolamento di rete garantisce la risoluzione DNS interna tramite il nome del servizio (es. `registry-node-1` risolve l'IP interno nella sottorete Docker). Inoltre, ogni nodo n_i monta un volume logico isolato (`docker-compose.yml:54`) associato alla directory interna `/app/data`, impedendo la collisione dei file di snapshot gRPC. Il `service-cli`, definito con il profilo `tools` (`docker-compose.yml:46`), non possiede un ciclo di vita continuo (`no-daemon`) ma viene invocato *on-demand*.

C. Pipeline di Compilazione Immagini (Multi-Stage Build)

Per minimizzare la superficie d'attacco e ottimizzare l'efficienza di deployment, sia il *Registry* sia la *Service CLI* utilizzano una strategia di compilazione **Multi-Stage** divisa in due stadi:

- 1) **Stage_{Build} (Compilazione):** Ambiente basato su immagine ufficiale `golang:alpine`. Vengono installate le dipendenze di build, scaricati i moduli e compilato il binario statico isolando le architetture hardware target:

$$\text{CGO_ENABLED} = 0 \quad \text{GOOS} = \text{linux} \quad \text{go build} -o \dots \quad (4)$$

- 2) **Stage_{Runtime} (Esecuzione):** Immagine di base `alpine:latest`. Vengono importati esclusivamente gli artefatti strettamente necessari all'esecuzione runtime, riducendo radicalmente l'impronta sul disco del container. Nel caso del Registry (`Dockerfile.registry`), l'entrypoint definisce il caricamento del file di configurazione specifico passato via flag: `ENTRYPOINT ["registry"]` con default `CMD ["-config", "/app/config/registry/node-1.yaml"]`.

D. Bootstrapping Interno e Ciclo di Vita del Processo Go

All'avvio del binario Go dentro il container, il ciclo di vita del processo si articola in quattro fasi sequenziali a tolleranza d'errore:

- 1) **Parsing della Configurazione:** Il flag `-config` viene valutato per mappare la struttura dati YAML (`main.go:22`).
- 2) **Inizializzazione di Rete e Storage:** Viene aperto un listener TCP sulla porta designata e viene verificata la presenza di snapshot storici (`storage_dir`) per ripristinare lo stato in memoria prima di accettare traffico.
- 3) **Registrazione dei Server gRPC:** Vengono istanziati contemporaneamente i due endpoint logici del server (`main.go:70-71`): *Client API* (per operazioni esterne) e *Peer Gossip API* (per sincronizzazione interna).
- 4) **Attivazione dei Runtime Asincroni:** Vengono spawndate le goroutine dedicate al motore di Gossip (`main.go:73`) e al ticker di *Heartbeat Sweep* (`main.go:85`), il quale invalida i servizi con TTL scaduto ($t > \Delta t_{\text{stale}}$).

Alla ricezione di segnali POSIX (SIGINT o SIGTERM), il processo intercetta la notifica (`main.go:121`), interrompe i loop di ricezione del gossip, invia un pacchetto di *Graceful Leave* ai peer del cluster per notificare la propria disconnessione controllata, e infine arresta il server gRPC in modo pulito.

E. Topologia Gossip e il Meccanismo del Gossip Gate

La configurazione dei nodi governa il comportamento del protocollo di gossip e i limiti del cluster di rete tramite i parametri `listen_address` (binding interno, es. `0.0.0.0:50051`), `advertise_address` (hostname DNS esposto ai peer, es. `registry-node-1:50051`) e `seed_peers` (array di bootstrap per l'aggancio iniziale alla rete mesh):

$$P_{\text{seeds}}(n_1) = \{"registry-node-2:50051", "registry-node-3:50051"\} \quad (5)$$

Per evitare condizioni di *race condition* durante il cold bootstrap del cluster, il runtime di Go implementa una barriera di sincronizzazione logica chiamata **Gossip Gate** (`runtime.go:246`). Sia G_i lo stato del gate del nodo i -esimo ed E l'evento di esistenza del file di controllo:

$$G_i = \begin{cases} \text{BLOCKED}, & \neg \exists (\text{gossip_gate_path}) \\ \text{UNBLOCKED}, & \exists (\text{gossip_gate_path}) \end{cases} \quad (6)$$

Tutti i loop fondamentali del sistema d'orchestrazione distribuiti (`bootstrap`, `gossip`, `reconcile` e `peer sweep`) eseguono un'attesa bloccante sul gate:

$$\forall \text{loop} \in \{\text{Gossip}, \text{Reconcile}, \text{Sweep}\}, \quad \text{Attendi}(G_i == \text{UNBLOCKED}) \quad (7)$$

La transizione di stato da BLOCKED a UNBLOCKED avviene programmaticamente tramite l'esecuzione di

`make trace-up`, il quale esegue un comando di `touch .gossip-enabled (trace-register)` sui tre nodi, sbloccando l'esecuzione dei processi in modo sincrono.

F. Esecuzione dei Comandi tramite CLI Tool Effimera

Il container `service-cli` interagisce con il cluster iniettando comandi atomici all'interno della rete mesh di Docker tramite il costrutto `docker compose run -rm service-cli [subcommand]`. L'argomento `-rm` garantisce l'eliminazione del container ($\mathcal{T}_{\text{container}} \rightarrow \emptyset$) al termine del processo, azzerando i residui sull'host.

I comandi mappati all'interno di `main.go:37` rispondono all'insieme:

$$\text{Subcommands} = \{\text{register, heartbeat, deregister, list}\} \quad (8)$$

In caso di partizionamento di rete o di spegnimento di uno o più nodi del cluster, la subcommand `list` implementa un pattern di **Degradazione Controllata (Graceful Degradation)** (`main.go:148`). Invece di sollevare un'eccezione non gestita (*panic*), il client intercetta l'errore di connessione gRPC ed evidenzia l'anomalia visualizzando la stringa stringente "**nodo disattivato**".

G. Modello di Persistenza e Mutazione dello Stato

La persistenza dello stato globale del registro è implementata tramite un meccanismo di **Event-Driven Snapshotting** su file system locale nel file `JSON registry_snapshot.json (persistence.go:15)`.

All'avvio, il nodo invoca `load()` (`persistence.go:22`) eseguendo una transizione $S_0 \leftarrow D_{\text{disk}}$. Al verificarsi di una mutazione (es. una nuova registrazione tramite la CLI), lo stato in memoria viene aggiornato e viene triggerata una persistenza immediata su disco:

$$S_{t+1} = \delta(S_t, \text{evento}) \implies \text{save}(S_{t+1}) \rightarrow D_{\text{disk}} \quad (9)$$

La persistenza dei dati sul disco host varia radicalmente a seconda del target di terminazione invocato dall'operatore, come descritto nella Tabella I.

Tabella I
EFFETTI DEI COMANDI DI ARRESTO SULLA PERSISTENZA DEI DATI.

Target	Comando Sottostante	Impatto sullo Stato e sui Volumi
<code>make compose-down</code>	<code>docker compose down</code>	Conservativo: I container vengono arrestati, ma i volumi associati a <code>/data</code> rimangono intatti sull'host. Al successivo avvio lo stato viene ripristinato.
<code>make trace-up</code>	<code>docker compose down -v</code>	Distruttivo: Il flag <code>-v</code> impone la distruzione dei volumi. Lo snapshot viene eliminato, azzerando la memoria storica per un <i>cold start</i> pulito.

H. Inizializzazione del nodo e ciclo di vita (Lifecycle)

L'architettura software del singolo nodo registry è progettata come un sistema a componenti reattivi guidati dagli eventi e da temporizzatori asincroni. La gestione del ciclo di vita del processo Go, dall'istante di avvio (*bootstrap*) fino allo spegnimento controllato (*graceful shutdown*), segue un flusso sequenziale rigoroso e deterministico, volto a garantire l'integrità dei dati e la coerenza della topologia del cluster.

1) **Fase di Bootstrapping e Inizializzazione:** All'atto dell'esecuzione del binario compilato, il processo esegue i seguenti passi fondamentali in ordine sequenziale:

- 1) **Parsing della Configurazione:** Il nodo effettua il caricamento e la validazione di un file di configurazione in formato **YAML**, mappandolo su una struttura Go strongly-typed. I parametri essenziali estratti includono:
 - **node_id:** Stringa univoca identificativa del nodo all'interno del cluster.
 - **bind_address** e **peer_address:** Gli endpoint di rete per l'esposizione dei servizi gRPC pubblici e interni.
 - **seed_peers:** Una lista di indirizzi di nodi noti stabili, utilizzati come punto di ingresso per aggregarsi al cluster.
 - **cluster_params:** Parametri di tuning del protocollo, quali l'intervallo di gossip (*Gossip Interval*) e il fattore di fan-out.
 - **heartbeat_ttl:** Il Time-To-Live massimo oltre il quale un microservizio registrato viene considerato orfano o non responsivo.
- 2) **Allocazione delle Strutture Dati in Memoria:** Vengono istanziati i moduli **ServiceStore** e **PeerStore**. Entrambi i componenti allocano le proprie tabelle hash interne (**map** di Go) e configurano le rispettive primitive di sincronizzazione della concorrenza (**sync.RWMutex**) necessarie a isolare i thread di lettura e scrittura.
- 3) **Ripristino dello Stato (State Restore):** Se la direttiva di configurazione **storage_dir** è valorizzata ed è presente uno snapshot valido sul volume persistente, il motore di storage avvia la procedura di ripristino. Il file viene deserializzato in memoria, consentendo al **ServiceStore** di ricostruire l'ultima vista valida nota prima dell'ultimo spegnimento, riducendo l'overhead di rete durante la successiva fase di sincronizzazione.
- 4) **Auto-Registrazione:** Il nodo inserisce le proprie informazioni identificative (ID, indirizzo gRPC, stato attivo) all'interno del proprio **PeerStore**, marcando se stesso come primo membro della propria vista di appartenenza al cluster.
- 5) **Attivazione dei Runtime Asincroni e dei Ticker:** Viene avviato il server gRPC principale su una *goroutine* dedicata per non bloccare il

thread principale. Simultaneamente, vengono spawmati i cicli di background (*loops*) governati da oggetti `time.Ticker` di Go. Tra questi, assume rilevanza critica il **ticker di stale detection**, il quale scatta a intervalli regolari per scansionare il `ServiceStore` ed eliminare o marcare come *Stale* (scaduti) i record dei servizi il cui `last_heartbeat_timestamp` è inferiore alla soglia determinata dal `heartbeat_ttl`.

2) Fase di Shutdown Controllato (*Graceful Leave*):

Per evitare la persistenza di record fantasma nei peer e facilitare la convergenza immediata della topologia di rete senza attendere i timeout di background, il processo intercetta i segnali di terminazione del sistema operativo (es. `SIGINT` o `SIGTERM`) tramite il pacchetto `os/signal` di Go.

All'intercettazione del segnale, viene avviata la routine di shutdown strutturato:

- 1) **Interruzione dei Loop Periodici:** Attraverso l'uso dei contesti nativi di Go (`context.Context`) e la chiusura di canali di controllo (*cancellation channels*), vengono interrotti in modo pulito tutti i ticker attivi (gossip, stale detection, anti-entropia), impedendo l'invio di nuove richieste di rete.
- 2) **Notifica di Graceful Leave:** Il nodo effettua una serie di chiamate gRPC sincrone verso i peer attivi presenti nel suo `PeerStore`, notificando la propria intenzione di abbandonare il cluster (*leave message*). Al ricevimento di questo messaggio, i peer aggiornano immediatamente lo stato di quel nodo in `Dead` o lo rimuovono dalla lista di gossip senza attendere che scatti la rilevazione per assenza di risposta.
- 3) **Arresto del Server gRPC:** Infine, viene invocato il metodo `GracefulStop()` sul server gRPC. Questa primitiva blocca l'accettazione di nuove connessioni, attende l'evasione di tutte le RPC pendenti attualmente in fase di elaborazione e, solo successivamente, rilascia i socket di rete e termina definitivamente il processo Go con codice di uscita zero.

I. Gestione dello stato dei servizi e mutazioni di stato

Il modulo `ServiceStore` agisce come una macchina a stati finiti in-memory per ciascun microservizio registrato. Al fine di garantire la coerenza eventuale in un ambiente asincrono, ogni mutazione dello stato locale deve seguire regole rigorose di validazione e versionamento, minimizzando il rischio di corruzione dei dati o di inversione temporale degli aggiornamenti durante le fasi di gossip.

1) **Procedura di Registrazione (*Registration*):** Quando una nuova istanza di microservizio invia una richiesta tramite l'RPC `RegisterService`, il nodo ricevente agisce come gateway ed esegue le seguenti operazioni:

- **Validazione dei Vincoli:** Viene verificata la presenza dei campi obbligatori nel payload (`service_name`,

`service_id`, `endpoint` e `version`). Se uno di questi elementi risulta malformato o assente, la richiesta viene respinta sollevando un errore gRPC di tipo `InvalidArgument`.

- **Inizializzazione dello Stato:** Al record viene assegnato lo stato di salute iniziale di default `SERVING` (o `HEALTHY`) e viene campionato il timestamp corrente per il campo `last_heartbeat_timestamp`.
- **Gestione del Versionamento Logico:** Il nodo consulta la mappa interna del `ServiceStore`:
 - Se la coppia `service_name/service_id` non è presente, il record viene creato *ex-novo* con una `logical_version` pari a 1.
 - Se il servizio è già censito (es. un tentativo di registrazione o sovrascrittura), il nodo incrementa monotonamente la `logical_version` precedente (`versione = versione + 1`), garantendo che questa mutazione prevalga sulle repliche distribuite antecedenti.

2) **Meccanismo di Rinnovo (*Heartbeat*):** Il mantenimento della presenza nel registro richiede un flusso continuo di keep-alive da parte del microservizio. Al ricevimento di un'invocazione `Heartbeat`, il nodo verifica la preesistenza dell'identificativo nel proprio store locale. L'aggiornamento del campo `last_heartbeat_timestamp` e la conferma dello stato `SERVING` avvengono ***condizionatamente***: la coppia identificativa deve esistere e non deve essere marcata come eliminata (*tombstone*). Qualora il servizio fosse sconosciuto o già rimosso dal cluster, l'heartbeat viene esplicitamente rifiutato con un codice di errore, forzando il client a eseguire una nuova procedura di registrazione completa.

3) **Cancellazione Logica tramite Tombstone:** In un sistema distribuito decentralizzato, l'eliminazione fisica immediata di un record da un nodo locale è un'operazione deleteria: se il nodo *A* elimina un record e successivamente scambia i dati via gossip con il nodo *B* (che possiede ancora quel record), il nodo *A* re-imparerà erroneamente l'esistenza del servizio eliminato, un fenomeno noto come *phantom update*.

Per ovviare a questo problema, il progetto implementa il pattern della **Tombstone** (cancellazione logica):

- All'atto della `DeregisterService`, i metadati operativi del record (`endpoint` e `version`) vengono svuotati o azzerati.
- Lo stato viene impostato permanentemente su `NOT_SERVING` (o `DELETED`).
- La `logical_version` viene categoricamente incrementata.

In questo modo, la cancellazione si propaga sulla rete sotto forma di aggiornamento ad alta priorità. I peer che ricevono la tombstone via gossip noteranno la versione logica superiore e applicheranno la cancellazione logica anche sui propri store. La rimozione fisica definitiva del record (*garbage collection*) avverrà solo dopo un periodo di tempo

sufficientemente lungo da garantire che la tombstone abbia raggiunto l'intero cluster.

4) *Rilevazione Automatica delle Scadenze (Stale Detection)*: Per gestire i crash non controllati dei microservizi (in cui l'istanza smette di funzionare senza poter invocare la deregistrazione), il modulo runtime esegue un ciclo di **Stale Detection** guidato da un ticker periodico indipendente.

Il processo esegue una scansione lineare del **ServiceStore** calcolando il delta temporale per ciascun servizio attivo:

$$\Delta t = t_{\text{current}} - t_{\text{last_heartbeat}}$$

Se $\Delta t > \text{TTL}$, il servizio viene considerato instabile o spento. Il nodo non elimina il record, ma applica localmente la stessa logica della tombstone: incrementa la **logical_version** e muta lo stato in **NOT_SERVING**. Sebbene questo rilevamento avvenga su base puramente locale (poiché guidato dall'orologio del nodo proprietario o del nodo che rileva lo stale), l'incremento della versione logica fa sì che l'informazione di obsolescenza venga distribuita in modo deterministico a tutto il cluster tramite i successivi cicli di gossip, portando alla convergenza dello stato globale.

J. Protocollo di replica gossip e riconciliazione dello stato

Il cuore della decentralizzazione del sistema risiede nel modulo **Runtime Gossip**. Per garantire la convergenza dello stato verso una consistenza eventuale senza gravare sulla rete con protocolli di consenso sincroni e bloccanti, il runtime esegue in parallelo quattro cicli di background (*loops*) asincroni. Ognuno di essi è governato da un proprio timer indipendente e gestisce un aspetto specifico della sincronizzazione o della gestione della topologia.

1) I Quattro Loop di Background del Runtime:

- **Bootstrap Loop (Cluster Discovery)**: Operando a intervalli regolari (specialmente all'avvio o in caso di isolamento), questo ciclo invia richieste **JoinCluster** verso l'elenco dei *seed peers* configurati nello YAML. Lo scopo è duplice: annunciare la propria presenza per essere inseriti nelle tabelle di routing dei peer stabili e ricevere in risposta la lista aggiornata dei nodi attualmente attivi nel cluster, popolando dinamicamente il **PeerStore** locale oltre la configurazione statica iniziale.
- **Gossip Loop (Push Epidemico / Rumor Mongering)**: È il motore primario di diffusione rapida delle novità. A intervalli ad alta frequenza (es. ogni 500 ms), il nodo seleziona in modo pseudo-casuale un sottoinsieme ristretto di destinatari dal proprio **PeerStore**, definito dal parametro di *fan-out* (k). Verso questi nodi invia un messaggio **GossipSync** contenente gli ultimi aggiornamenti del proprio **ServiceStore**. La propagazione segue un modello epidemico altamente resiliente: la probabilità che un'informazione non

raggiunga l'intero cluster decada esponenzialmente al crescere dei cicli di trasmissione.

- **Reconcile Loop (Anti-Entropy Pull)**: Mentre il gossip push è ottimizzato per la velocità, il ciclo di riconciliazione garantisce la robustezza a lungo termine contro la perdita di pacchetti o il partizionamento della rete. Eseguito a intervalli più lunghi (es. ogni 10 s), il nodo sceglie un peer attivo e avvia uno scambio in modalità *pull* invocando la **RPC PullState**. La richiesta include il parametro **since_unix**, una marcatura temporale che indica l'ultimo allineamento eseguito. Il peer risponde inviando in modo incrementale solo i record modificati o creati successivamente a quel timestamp, riducendo drasticamente l'uso della banda rispetto a un trasferimento totale dello snapshot.
- **Peer Sweep Loop (Membership Pruning)**: Questo ciclo monitora la salute interna del cluster stesso. Scansiona periodicamente il **PeerStore** e calcola il tempo trascorso dall'ultima interazione gRPC riuscita con ciascun peer. Se un nodo non risponde ai messaggi o non invia alcun segnale per un tempo superiore alla soglia di tolleranza preimpostata, il suo stato viene mutato in **Suspect** e, successivamente, rimosso dal pool dei nodi attivi (*eviction*), isolando i nodi permanentemente spenti dal flusso del gossip.

2) *Meccanismi Sinergici per la Convergenza*: La consistenza eventuale dello stato globale del registro non è delegata a un singolo algoritmo, ma si ottiene mediante la combinazione sinergica e matematica di tre strategie complementari orientate a massimizzare la tolleranza ai guasti:

- 1) **Propagazione Epidemica Proattiva (Gossip Push)**: Garantisce che ogni nuova mutazione di stato (come una registrazione o l'applicazione di una *tombstone*) si diffonda nel cluster in tempo sub-lineare ($O(\log N)$, dove N è il numero di nodi). È la prima linea di difesa per mantenere i registri allineati in condizioni operative normali.
- 2) **Riallineamento Incrementale Reattivo (Reconcile Pull)**: Agisce come un meccanismo di *anti-entropia*. Colma i gap informativi causati da congestione di rete, messaggi gRPC persi o dal riavvio di un nodo dopo un crash prolungato. L'approccio *pull* incrementale garantisce che, anche se un nodo perde la fase di push epidemico, recupererà lo stato corretto al successivo ciclo di riconciliazione.
- 3) **Ri-bootstrap Periodico su Nodi Seme (Seed Resiliency)**: Impedisce la frammentazione del cluster in sotto-reti isolate (fenomeno del *network split*). Nel caso in cui un partizionamento transitorio della rete causi l'azzeramento o l'obsolescenza del **PeerStore** locale, il riesame ciclico dei *seed*

peers permette ai nodi di ricollegarsi alla spina dorsale stabile del cluster, innescando nuovamente il recupero dello stato.

K. Politica di merge e risoluzione deterministica dei conflitti

La natura asincrona e decentralizzata del protocollo di gossip implica che i nodi ricevano costantemente vettori di stato da peer differenti, potenzialmente disordinati, duplicati o ritardati a causa delle latenze di rete (*out-of-order delivery*). Per garantire che tutti i nodi convergano verso la medesima vista logica senza l'ausilio di un coordinatore centrale o di lock distribuiti, il sistema applica una politica di **merge deterministica e isolata** sul singolo **ServiceRecord**.

La funzione di merge agisce come una funzione di commutazione parziale che deve soddisfare le proprietà algebriche di un semireticolo superiore (*bounded join-semilattice*): deve essere ****idempotente**** ($A \sqcup A = A$), ****commutativa**** ($A \sqcup B = B \sqcup A$) e ****associativa**** ($(A \sqcup B) \sqcup C = A \sqcup (B \sqcup C)$).

Quando un nodo riceve un record remoto R_{remote} per un servizio identificato dalla coppia **service_name/service_id**, lo confronta con il corrispondente record locale R_{local} applicando una cascata di regole gerarchiche e strettamente deterministiche:

- 1) **Priorità Assoluta alla Versione Logica (logical_version)**: Il primo livello di sbarramento valuta il contatore monotono del record. L'aggiornamento viene accettato se e solo se la versione remota è strettamente maggiore di quella locale:

Se $R_{\text{remote}}.\text{logical_version} > R_{\text{local}}.\text{logical_version}$

Questo meccanismo cattura la causalità logica delle mutazioni (registrazioni, deregistrazioni, cambi di stato) indipendentemente dal tempo fisico. Se la versione remota è minore, il pacchetto viene scartato poiché considerato obsoleto.

- 2) **Primo Tie-Break - Timestamp di Mutazione (updated_at_unix)**: Qualora le versioni logiche coincidano ($R_{\text{remote}}.\text{logical_version} == R_{\text{local}}.\text{logical_version}$), si rende necessario risolvere un potenziale conflitto di modifiche concorrenti basandosi sul tempo fisico. Viene confrontato il timestamp di ultima modifica della struttura dati (**updated_at_unix**), adottando la politica del *Last-Write-Wins* (LWW): prevale il record che presenta la marcatura temporale Unix più recente.
- 3) **Secondo Tie-Break - Timestamp di Vitalità (last_heartbeat_unix)**: In caso di ulteriore parità (stessa versione logica e stesso timestamp di mutazione), il sistema analizza il timestamp dell'ultimo segnale di keep-alive ricevuto dal microservizio (**last_heartbeat_unix**). Il record con l'heartbeat

più recente viene considerato più aggiornato rispetto allo stato di salute effettivo dell'istanza e viene eletto come nuovo stato locale.

- 4) **Tie-Break Finale su Campi Strutturali**: Nell'altamente improbabile scenario in cui tutti i parametri quantitativi e temporali precedentemente elencati risultino identici, il sistema ricorre a una regola di fallback puramente deterministica per evitare l'indeterminatezza algoritmica. Si esegue un confronto lessicografico o binario sequenziale sui restanti campi strutturali del payload, nell'ordine: **health_status**, **owner_node_id**, **endpoint** e **version**. Questo garantisce che, indipendentemente dal nodo in cui viene eseguito il merge, la scelta finale ricada in modo univoco e identico sullo stesso record.

Attraverso questa rigorosa gerarchia decisionale, l'operazione di convergenza non richiede alcuna forma di negoziazione tra i nodi del cluster. La convergenza finale dello stato tra nodi registry isolati è garantita matematicamente non appena tutti i messaggi di gossip incrociati completano la loro propagazione nella rete di trasporto.

L. Modello di persistenza locale e salvataggio atomico

Sebbene il registro dei servizi operi prevalentemente in memoria per garantire risposte a bassissima latenza alle richieste di discovery, la tolleranza ai guasti richiede una strategia di durabilità dei dati. Il sistema implementa un meccanismo di persistenza locale basato su ****snapshot JSON periodici ed espliciti****, memorizzati all'interno di una directory mappata su un volume Docker dedicato e persistente per ciascun singolo nodo.

1) **Scrittura Atomica sul File System (Double-Buffering Pattern)**: Per prevenire la corruzione dei dati causata da crash improvvisi del processo o del container durante la fase stessa di scrittura su disco (che lascerebbe il file JSON in uno stato parziale o malformato), il motore di storage adotta una strategia di scrittura atomica in due fasi, mutuata dai pattern operativi dei *file system POSIX-compliant*:

- 1) **Scrittura su File Temporaneo**: All'attivazione del trigger di persistenza (guidato da una mutazione dello stato o da un timer di background), il **ServiceStore** viene bloccato in lettura. Il payload complessivo dello store viene serializzato in formato JSON e riversato all'interno di un file temporaneo univoco (es. **registry_snapshot.tmp**) allocato nella stessa directory di destinazione.
- 2) **Sostituzione Atomica via Rename**: Una volta completata con successo la scrittura e flushato il buffer sul disco, il runtime esegue la chiamata di sistema di rinomina (**os.Rename** in Go, che si mappa sulla *syscall* atomica **rename(2)** del kernel Linux). Il file temporaneo va a sovrascrivere istantaneamente il file di produzione definitivo denominato **registry_snapshot.json**.

Grazie a questa atomicità a livello di kernel, il file di snapshot consolidato passa istantaneamente dalla versione precedente a quella successiva. Se il nodo subisce un crash distruttivo esattamente durante la scrittura, al riavvio il file temporaneo incompleto verrà semplicemente ignorato o rimosso, mentre il file `registry_snapshot.json` originale rimarrà integro e leggibile.

2) *Flusso di Riavvio e Abbattimento del Warm-Up Time*: La persistenza locale altera radicalmente il comportamento del nodo in fase di ripartenza post-crash, ottimizzando la metrica del *Recovery Time Objective* (RTO):

- **Fase di Cold-Start (Senza Persistenza)**: Un nodo riavviato partirebbe con uno store vuoto. Dovrebbe attendere passivamente i cicli di gossip dei peer o forzare un pesante *pull state* totale per ricostruire l'intera tabella dei servizi, generando un picco di traffico di rete e un transitorio di indisponibilità locale delle API (fase di *warm-up*).
- **Fase di Fast-Boot (Con Persistenza Attiva)**: Durante l'inizializzazione, il nodo rileva il file `registry_snapshot.json`, lo deserializza e ripopola istantaneamente il **ServiceStore** locale con l'ultimo stato noto pre-crash. Sebbene questa vista possa essere parzialmente obsoleta (mancando delle mutazioni avvenute durante il downtime del nodo), essa costituisce una base di partenza solida ad alta fedeltà.

Immediatamente dopo il caricamento dello snapshot, il runtime attiva i loop di **Gossip** e **Reconcile**. Il nodo scambia i propri vettori di versione con i peer superstiti: i meccanismi di anti-entropia dovranno veicolare sulla rete solo le variazioni incrementali accumulate durante il periodo di assenza. Questo approccio ibrido (stato locale + delta distribuito) riduce a frazioni di secondo il tempo di warm-up del registro, minimizzando l'impatto operativo sul cluster.

V. DEPLOYMENT E OPERATIVITÀ DELL'INFRASTRUTTURA

A. Containerizzazione e orchestrazione locale

Per garantire l'isolamento dei processi, la riproducibilità dell'ambiente di esecuzione e la simulazione realistica di un cluster di calcolo distribuito, l'intera infrastruttura è containerizzata e orchestrata tramite **Docker Compose**. Il file di specifica `docker-compose.yml` formalizza la topologia della rete e astrae la complessità del deployment locale, definendo in modo dichiarativo un ecosistema composto da tre nodi registry cooperanti e da un'utilità di controllo.

1) *Configurazione e Anatomia dei Nodi Registry*: I tre servizi di calcolo (convenzionalmente denominati **registry-1**, **registry-2** e **registry-3**) sono istanziati a partire dalla medesima immagine applicativa standard, generata tramite un processo di build multi-stage in Go. Ciascun container viene configurato in modo isolato agendo sui seguenti canali operativi:

- **Iniezione della Configurazione Dedicata**: Ogni istanza riceve un file di configurazione YAML specifico (o variabili d'ambiente equivalenti) tramite il meccanismo dei *bind mounts* o dei volumi di Docker. Questo permette di assegnare a ciascun container un `node_id` univoco e di istruire il runtime su quali siano gli indirizzi di rete dei peer stabili per la fase di bootstrap.
- **Mappatura delle Porte Host (Port Forwarding)**: All'interno della rete virtuale Docker, tutti i nodi possono ascoltare sulla medesima porta gRPC standard (es. la porta 50051). Tuttavia, per consentire l'interazione diretta da parte del sistema ospite (*Host OS*) o di tool di debug esterni, le porte dei container vengono esposte e mappate su porte distinte e sequenziali dell'host:
 - **registry-1** → porta host 50051
 - **registry-2** → porta host 50052
 - **registry-3** → porta host 50053
- **Isolamento dello Stato tramite Volumi Dedicati**: Per validare l'efficacia dei meccanismi di persistenza locale e prevenire conflitti di sovrascrittura sul file system, Docker Compose alloca tre volumi logici denominati e indipendenti (es. **registry_data_1**, **registry_data_2**, **registry_data_3**). Ogni volume viene montato internamente al rispettivo container in corrispondenza della directory di storage definita nello YAML (**storage_dir**). Questo garantisce che il ciclo di vita dei dati persistiti sia disaccoppiato da quello del container: un comando di **docker compose down** seguito da un **up** non causerà la perdita degli snapshot JSON.

2) *Isolamento di Rete (Network Bridge)*: I container sono inseriti all'interno di una rete virtuale isolata di tipo **bridge** creata esplicitamente dal demone Docker. All'interno di questa subnet, il server DNS integrato in Docker consente la risoluzione dinamica dei nomi dei servizi. Di conseguenza, i nodi non hanno bisogno di conoscere preventivamente gli indirizzi IP statici dei peer, ma possono configurare il proprio **PeerStore** utilizzando direttamente gli hostname logici (es. `dns://registry-2:50051`).

3) *Il Tool di Supporto: Service CLI*: Insieme ai nodi del cluster, il file di orchestrazione definisce un servizio aggiuntivo denominato **service-cli**. Questo container, che condivide la medesima rete bridge dei registri, non ospita un demone persistente ma funge da tool *ad-hoc* a riga di comando.

Il client CLI è preposto a iniettare stimoli nel sistema e verificarne le risposte. Essendo interno alla rete virtuale, può connettersi a qualsiasi nodo del cluster ed eseguire chiamate gRPC simulate (come la registrazione di un microservizio di test o l'invio di heartbeat campionati) agendo da sonda operativa per raccogliere le metriche di convergenza e tolleranza ai guasti descritte negli scenari

dimostrativi.

B. Automazione del ciclo di vita e testing tramite Makefile

La gestione operativa di un sistema distribuito, composto da molteplici nodi cooperanti, interfacce di rete e contratti di serializzazione da compilare, introduce un'elevata complessità di coordinamento. Al fine di garantire la massima riproducibilità degli scenari di test, azzerare l'errore umano e fornire un'interfaccia di interazione unificata e dichiarativa, l'intero ciclo di vita del progetto è centralizzato e automatizzato mediante un file **Makefile** strutturato per l'utilità **GNU Make**.

L'automazione si articola su quattro macro-aree funzionali, mappate su target chiari e sequenziali:

- **Verifica della Qualità del Codice (Build, Test e Lint):** Il **Makefile** astrae i comandi della *toolchain* di Go. Mediante target dedicati (es. **make build**, **make test**), esegue la compilazione statica dei binari ottimizzati, l'esecuzione della suite di *unit testing* (sfruttando le primitive native del pacchetto **testing** di Go) e l'analisi statica del codice (*linting*). Quest'ultima fase assicura la conformità del codice agli standard idiomati del linguaggio e previene l'introduzione di potenziali *race conditions* o memory leak prima del deployment.
- **Compilazione dei Contratti di Rete (Protobuf Generation):** Per evitare il disallineamento dei tipi e delle interfacce gRPC, il target **make proto** automatizza l'invocazione del compilatore **protoc** e dei relativi plugin per Go (**protoc-gen-go** e **protoc-gen-go-grpc**). Il comando scansiona i file di specifica **.proto**, generando in modo deterministico il codice sorgente stub per i servizi **ServiceRegistry** e **RegistryPeer**, isolando lo sviluppatore dalla gestione manuale delle dipendenze di compilazione.
- **Orchestrazione Semplificata dell'Infrastruttura:** I comandi di gestione dei container Docker Compose vengono incapsulati in target ad alto livello come **make compose-up-d**, **make compose-down** e **make trace-up**. Questo approccio garantisce la corretta sequenzialità delle operazioni, ad esempio assicurando che i volumi Docker residui vengano interamente purgati tra una sessione di test e la successiva, garantendo così uno stato iniziale del cluster sempre pulito ed esente da contaminazioni (*idempotenza del deployment*).
- **Scripting di Scenario e Fault Injection (trace-cover):** Il punto di forza dell'automazione risiede nel target **make trace-cover**. Questo comando non si limita ad avviare i container, ma esegue uno script Bash o Go ad-hoc che agisce come un vero e proprio orchestratore di test end-to-end. Il target implementa tecniche di **Fault Injection** controllato:

- 1) Interroga i nodi tramite il container **service-cli** per verificare lo stato iniziale.
- 2) Inietta un fallimento simulato spegnendo bruscamente un container specifico (**docker stop**).
- 3) Monitora i log in tempo reale e interroga i nodi superstiti per calcolare analiticamente il tempo di convergenza e la latenza di riconciliazione del protocollo di gossip.
- 4) Riattiva il nodo interrotto e convalida la corretta esecuzione della procedura di *recovery*.

1) *Riproducibilità Accademica e Validazione in Sede d'Esame:* L'approccio basato su **Makefile** eleva il progetto dal punto di vista metodologico, trasformandolo in un laboratorio software autosufficiente e deterministico. L'astrazione totale dei comandi infrastrutturali garantisce che chiunque — sia in fase di sviluppo locale, sia in sede di valutazione accademica — possa riprodurre l'intero scenario di validazione distribuita con un singolo comando. Ciò dimostra come la complessità operativa dei sistemi decentralizzati possa essere efficacemente governata mediante una rigorosa cultura dell'automazione e del testing continuo.

VI. STRATEGIA DI TEST E VALIDAZIONE

A. Test unitari

Sono presenti test mirati su:

- validazione API service server;
- gestione heartbeat e transizione a **NOT_SERVING**;
- semantica di deregistrazione;
- comportamento runtime gossip su bootstrap, push e pull;
- store dei peer e meccanismi di pruning.

B. Test di integrazione multi-nodo

I test di integrazione verificano scenari end-to-end:

- 1) convergenza gossip tra tre nodi;
- 2) crash di un nodo e sua rimozione dalla peer view;
- 3) resume con persistenza e riallineamento dello stato;
- 4) propagazione della deregistrazione (tombstone convergence);
- 5) graceful leave senza attendere timeout peer.

La copertura funzionale e quindi coerente con i requisiti di robustezza richiesti in un registro distribuito.

VII. ANALISI CRITICA: PUNTI DI FORZA E LIMITI

A. Punti di forza

- Architettura semplice e modulare, adatta a scopi didattici e sperimentali.
- Assenza di single point of failure a livello di registry.
- Replica eventual-consistent con buona resilienza a fault transitori.
- Persistenza locale sufficiente per recovery rapido nei casi di crash singolo.

B. Limiti tecnici

- Consistenza solo eventuale: possibile lettura temporaneamente stale.
- Comunicazioni gRPC in chiaro (assenza TLS/mTLS) nel setup corrente.
- Nessun quorum o consenso forte; non adatto a requisiti strict-serializable.
- Mancano metriche/telemetria integrate per osservabilità di produzione.
- Il folder `cmd/demo-service` è vuoto: il progetto usa CLI per simulare il service lifecycle, ma non include un microservizio demo persistente completo.

C. Miglioramenti proposti

- 1) Introduzione di autenticazione e cifratura mTLS tra peer e client.
- 2) Aggiunta di metriche Prometheus (latenza gossip, convergenza, peer churn).
- 3) Garbage collection controllata delle tombstone.
- 4) Politiche di backoff e retry più evolute sulle connessioni peer.
- 5) Eventuale backend persistente condiviso (o consenso Raft) per modalità strong consistency opzionale.

VIII. CONCLUSIONI

Il progetto soddisfa in modo convincente gli obiettivi tipici di un registro distribuito in ambito didattico SDCC: discovery decentralizzata, propagazione dello stato, tolleranza a crash parziali, recovery e verifica sperimentale ripetibile.

Dal punto di vista universitario, il lavoro mostra una buona padronanza dei concetti di sistemi distribuiti applicati in pratica:

- separazione tra API utente e API inter-nodo;
- gestione del ciclo di vita dei servizi con heartbeat e TTL;
- convergenza tramite gossip + reconcile;
- ragionamento su conflitti, versionamento logico e tombstone.

In sintesi, la soluzione è adeguata per una discussione accademica perché unisce chiarezza architetturale, implementazione concreta e una strategia di test coerente con gli obiettivi di resilienza.